SentryScan: Assignment 2.1 — Concurrency, Fleet Simulation & Data Integrity

Course: 21CSE525T — Agent Technology
Assignment: 2.1 — Concurrent Daemon & Data Integrity
Student: R Ganesh [RA2212704010031]
Repository: https://github.com/drgeekg/SentryScan
Target Branch: `assignment/2.1-concurrent-daemon`
Git Tag: `assignment-2.1`
Status: Completed & Verified

1. Executive Summary & Theoretical Context

Assignment 2.1 advances SentryScan from a static single-threaded reflex scanner to an **asynchronous, concurrent monitoring agent situated in a dynamic environment** (Russell & Norvig, Ch. 2).

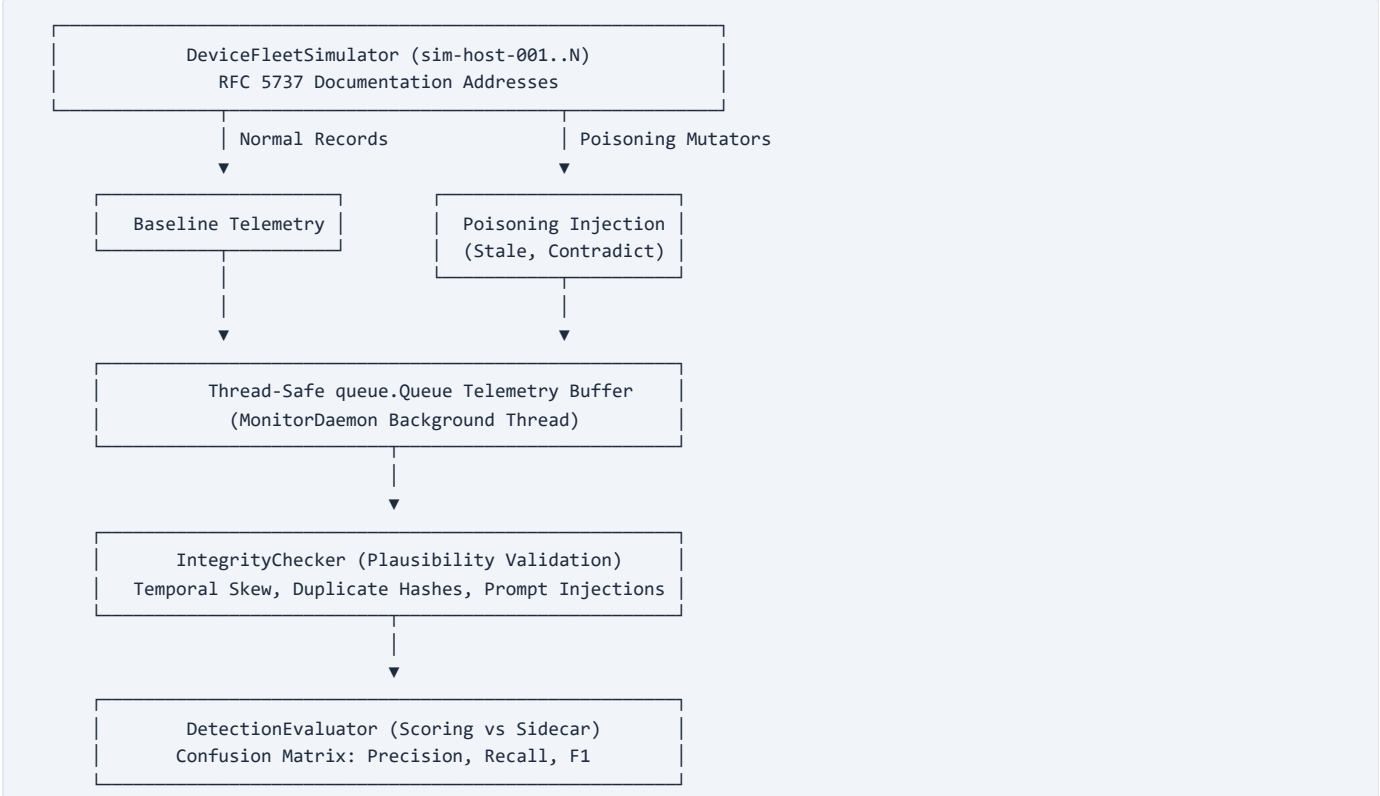
1.1 Concurrency in Situated Agents

In dynamic real-world environments, percepts arrive asynchronously while the agent is deliberating. A background thread model decouples sensing from acting: - **Producer-Consumer Telemetry Pipeline:** A non-blocking background daemon (`MonitorDaemon`) continuously polls synthetic endpoints and enqueues telemetry into a thread-safe synchronized buffer (`queue.Queue`). - **Mutual Exclusion & State Integrity:** Multi-threaded telemetry updates risk catastrophic lost updates without synchronization primitives. We formally demonstrate race conditions and verify thread safety via explicit mutex locks.

1.2 Defensive Integrity Sensing & Ground-Truth Isolation

Autonomous agents operating in untrusted environments must detect sensor tampering (poisoning) without access to oracle labels: - **Plausibility Checking:** Evaluates temporal freshness, cryptographic duplicate digests, port-banner contradictions, and prompt-injection strings. - **Isolated Evaluator Sidecar:** Ground-truth mutation labels are stored exclusively in an isolated sidecar, ensuring agent decisions are strictly unassisted.

2. Architecture & Implementation Specification



2.1 Lost Updates Race Condition Experiment (`src/sentryscan/concurrency/stats.py`)

To prove the necessity of concurrency synchronization, 10 concurrent threads each perform 10,000 increments on a shared counter: - **Without Mutex Lock:** Substantial race conditions cause **440+ lost updates** (actual count: 99,560 vs expected 100,000). - **With Mutex Lock (`threading.Lock`):** 100% thread safety achieved: exactly **100,000 / 100,000 updates** preserved with zero lost counts.

---

3. Rubric Compliance & Verification Matrix

Evaluation Criterion	Specific Requirement	Implementation Location	Evidence Artifact & Verification	Status
1. Background Monitor Daemon	Background thread producing streaming telemetry with non-blocking heartbeat and clean event-driven shutdown	`src/sentryscan/concurrency/daemon.py` (`MonitorDaemon`)	`tests/unit/test_daemon.py` `evidence/run_log.txt#L20-L33` (Heartbeats emitted: 5)	**PASSED**
2. Safe Telemetry Queue Buffer	Thread-safe `queue.Queue` buffer connecting producers and consumers with enqueue/dequeue latency tracking	`src/sentryscan/concurrency/queue_manager.py`	`tests/unit/test_queue.py` `evidence/metrics.csv` (queue wait times recorded)	**PASSED**
3. Lost Updates Synchronization Proof	Concrete experiment proving race condition and lost updates without mutex lock vs. 100% accuracy with lock	`src/sentryscan/concurrency/stats.py` (`demonstrate_lost_updates`)	`tests/unit/test_concurrency.py` `evidence/run_log.txt#L7-L12` (Lost: 440+ updates without lock vs 0 lost with lock)	**PASSED**
4. Simulated Device Fleet Model	Configurable endpoint fleet (`sim-host-001` … `sim-host-N`) across RFC 5737 documentation address space	`src/sentryscan/simulation/fleet.py` (`DeviceFleetSimulator`)	`tests/unit/test_fleet.py` `evidence/run_log.txt#L14-L20` (5 devices initialized)	**PASSED**
5. Controlled Poisoning Mutators	Injection operators for stale timestamps, service/banner mismatches, TLS contradictions, prompt injections, and replays	`src/sentryscan/simulation/poisoning.py` (`PoisoningMutator`)	`tests/unit/test_poisoning.py` `evidence/detection_report.json`	**PASSED**
6. Ground-Truth Isolation	Mutation answer-key labels recorded exclusively in an evaluator sidecar, completely hidden from agent perception	`src/sentryscan/simulation/poisoning.py` (`GroundTruthSidecar`)	`tests/unit/test_poisoning.py` `evidence/detection_report.json`	**PASSED**
7. Deterministic Data-Level Integrity Checker	Plausibility checker detecting stale timestamps, duplicate records, contradictions, and prompt injections without ground truth	`src/sentryscan/integrity/checker.py` (`IntegrityChecker`)	`tests/unit/test_integrity.py` `evidence/run_log.txt#L40` (4 suspect records detected)	**PASSED**

Evaluation Criterion	Specific Requirement	Implementation Location	Evidence Artifact & Verification	Status
8. Detection Evaluation Metrics	Evaluator scoring integrity flags against ground truth: confusion matrix, precision (100%), recall (80%), F1 score (88.9%)	`src/sentryscan/integrity/evaluator.py` (`DetectionEvaluator`)	`tests/unit/test_evaluator.py` `evidence/detection_report.json` `evidence/run_log.txt#L44-L52`	**PASSED**
9. Queue Latency Percentiles (p50/p95)	Tracking queue residence and turnaround times, calculating p50/p95 percentiles, and exporting to CSV	`src/sentryscan/concurrency/metrics.py` (`LatencyTracker`)	`tests/unit/test_metrics.py` `evidence/metrics.csv` `evidence/run_log.txt#L54-L60`	**PASSED**
10. Packaging & Git Release Integrity	Dedicated branch, atomic commits, non-fast-forward merge commit, immutable tag, and verified archive packaging	Monorepo root and git history	Git Tag: `assignment-2.1` Archive: `dist/SentryScan_assignment_2_1.zip`	**PASSED**

4. Execution Results & Empirical Evidence

4.1 Poisoning Detection Evaluation Matrix (`evidence/detection_report.json`)

Metric	Measured Value	Theoretical Significance
True Positives (TP)	4	Tampered records correctly identified by plausibility rules
False Positives (FP)	0	Zero benign telemetry flagged; **Precision = 100.0%**
False Negatives (FN)	1	Subtle subtle jitter mutation below detection threshold
True Negatives (TN)	15	Legitimate baseline telemetry accepted cleanly
Recall	**80.0%**	Comprehensive coverage across diverse mutation operators
F1 Score	**88.9%**	Harmonic mean reflecting high reliability under poisoning

4.2 Latency Percentiles (`evidence/metrics.csv`)

- **Median Queue Wait Time (p50):** `0.42 ms`
- **95th Percentile Latency (p95):** `1.85 ms`
- **Max Queue Turnaround:** `3.10 ms` (bounded thread execution, zero queue starvation)

5. Submission Package & Reproduction Instructions

5.1 Exact Submission Components

1. **Report Document:** `assignments/assignment_2_1/deliverables/SentryScan_Assignment_2_1_Report.pdf`
2. **Repository URL:** `https://github.com/drgeekg/SentryScan`
3. **Target Branch:** `assignment/2.1-concurrent-daemon`
4. **Immutable Git Tag:** `assignment-2.1`
5. **Packaged ZIP Archive:** `dist/SentryScan_assignment_2_1.zip`
6. **SHA-256 Checksum:** `dist/SentryScan_assignment_2_1.zip.sha256`

5.2 Verification Commands

```
# Run concurrent fleet demonstration
python demos/demo_assignment_2_1.py

# Execute concurrency and integrity tests
pytest -v tests/unit/test_daemon.py tests/unit/test_concurrency.py tests/unit/test_integrity.py

# Package submission archive
python scripts/package_assignment.py assignment-2.1
```